XGBoost Introduction

12 October 2023 09:38

ML → 70's 80's → linear → linear

Naive bays → text

→ 90's → RF, SVM, GB

Overfitting → performance

2000's

Scalability → speed

Xgboost

2014

Not an algorithm

library

Gradient Boosting

perfor

speed

ML + SE → library

gradien

Early Days → Kaggle pe bawaal → [Open source days]

→ (2014)

[Gradient Boostings]

- flexibity (Loss function)
  - reg
  - class
  - ranking
  - custom
- performance
- Robust
  - missing value
  - Regu
    - perform
    - big data
- kaggle → GB

(2016)

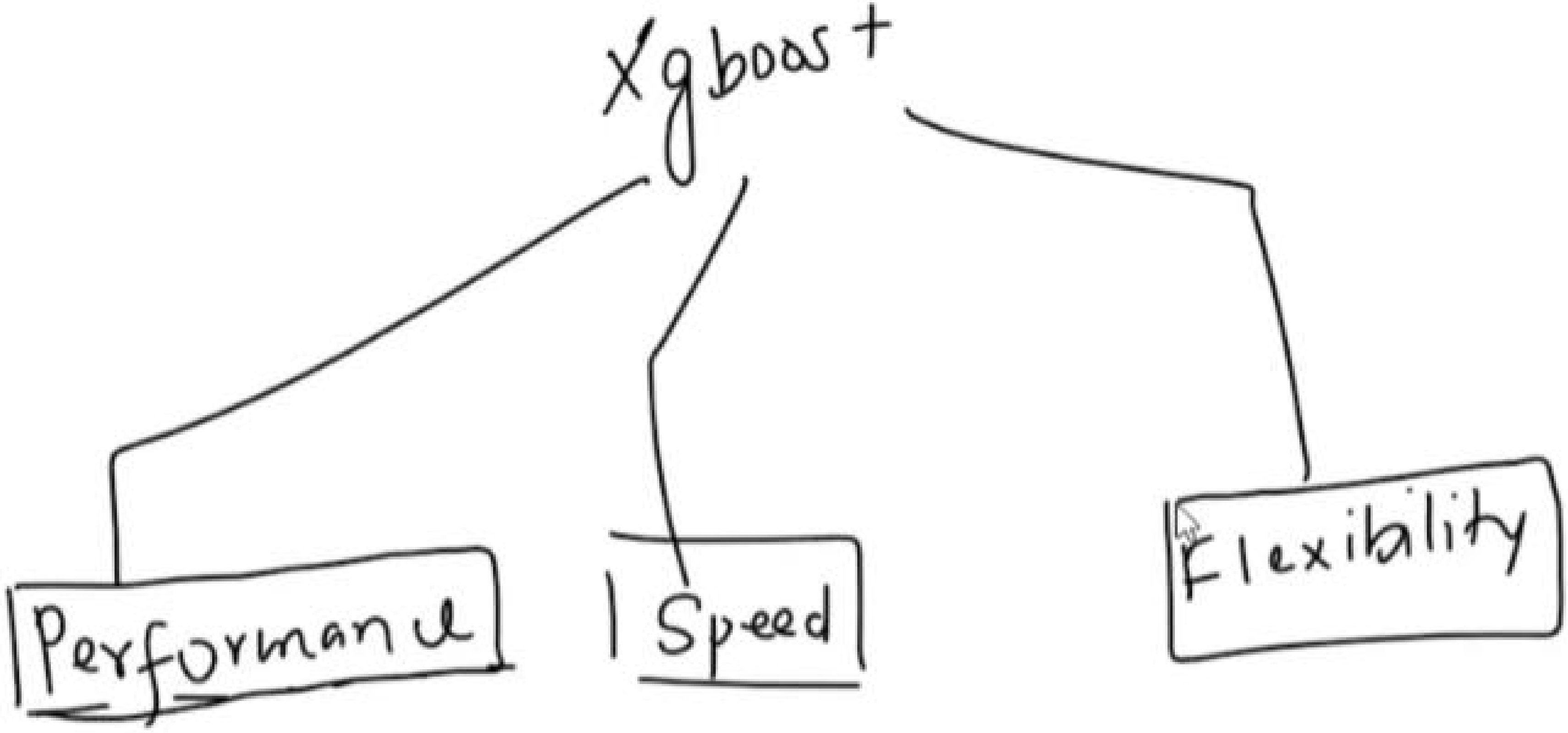
- → Higgs Boson comp (won)
- → 2011 → 29 winning
  - 16 → xgboost

Tianqi

- feature
- Optimiza
- multiple / prog
- documenta
- kaggle

XGBoost Features

12 October 2023 21:55



1. Flexibility

12 October 2023 22:01

1. Cross Platform → Linux, windows
2. Multiple Language Support → python, R, matplo / matlab
3. Integration with other libraries and tools
4. Support all kinds of ML problems

GBDT

Java → Scala, Rusy, python, R

loss function → loss

false posi / fan

+ve w
-ve

rank
recom

Java → python

python → api → java

linear
- regressi

Logist
- classi

Reg, Class, Time series, Ranking, Anamoly

binary, multi

fore

model building
- numpy / pandas
- scikit

distributed
- Spark / pysp
- Dask

model inter
SHAP
Lime

model deplo
docker
kubern

workflow man.
- Airflow
- mlflow

2. Speed

13 October 2023 20:55

1. Parallel Processing
2. Optimized Data Structures
3. Cache Awareness
4. Out of Core computing
5. Distributed Computing
6. GPU Support

Xgboost

200

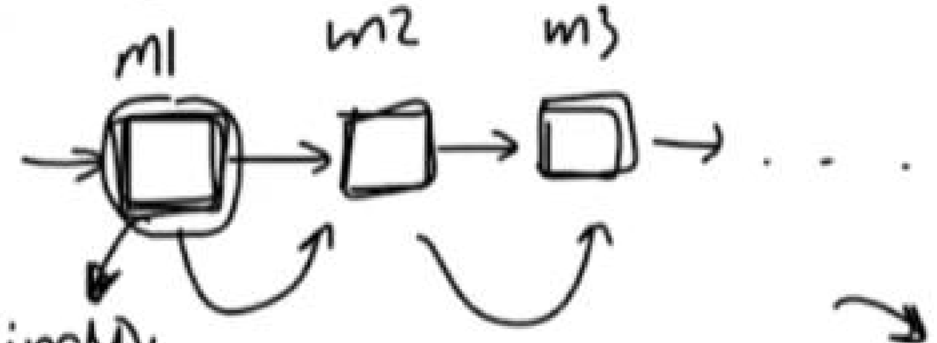
30 days (5)

6 days

8 cores

cores

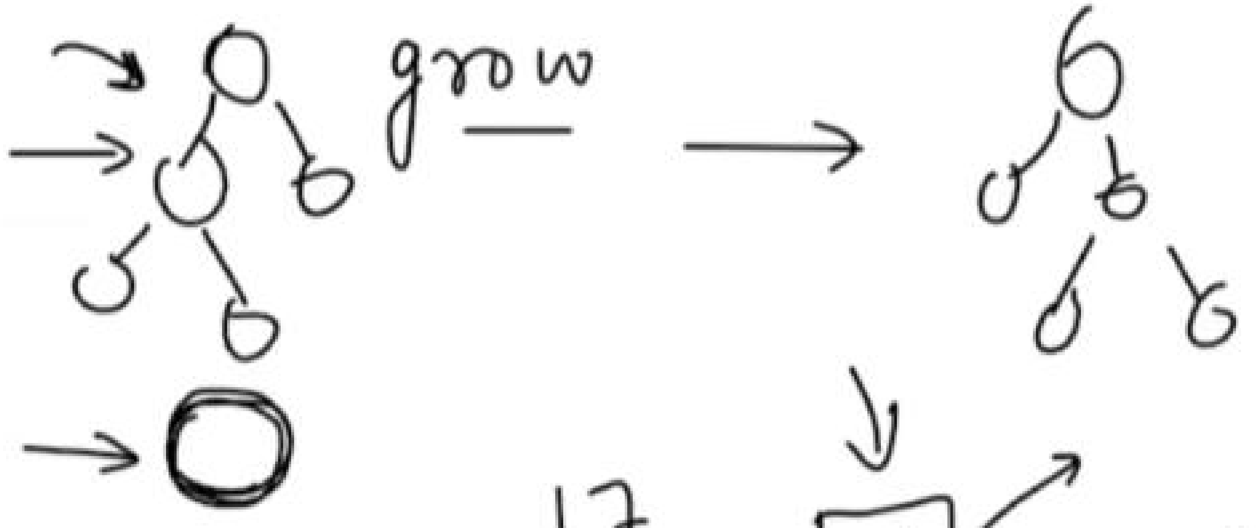
$\boxed{n\_jobs = -1}$



model building

sequential

grow



f_1	f_2	
age	marks	placement
21	98	13
19	89	11
17	95	15

17
19
21

17, 19 → 18 → entropy / gini

19, 21 → 20 →

spli

$f_1 > 18 \rightarrow$ gini

89
95
98

89, 95 → 92 gini

95, 98 → 97 gini

$f_2 > 97 \rightarrow$ gini

1. Parallel Processing
2. Optimized Data Structures
3. Cache Awareness
4. Out of Core computing
5. Distributed Computing
6. GPU Support

tree building

row wise

histogram

↓

bin

coloumn block

cgpa	iq	placement
7	70	7
8	80	8

row blocks

50 →
150

50-60 | 60-70 | . . .

hs

Cache memory

Col block → Col block

↳ splitting

4MB
8MB

CPU — RAM

cache

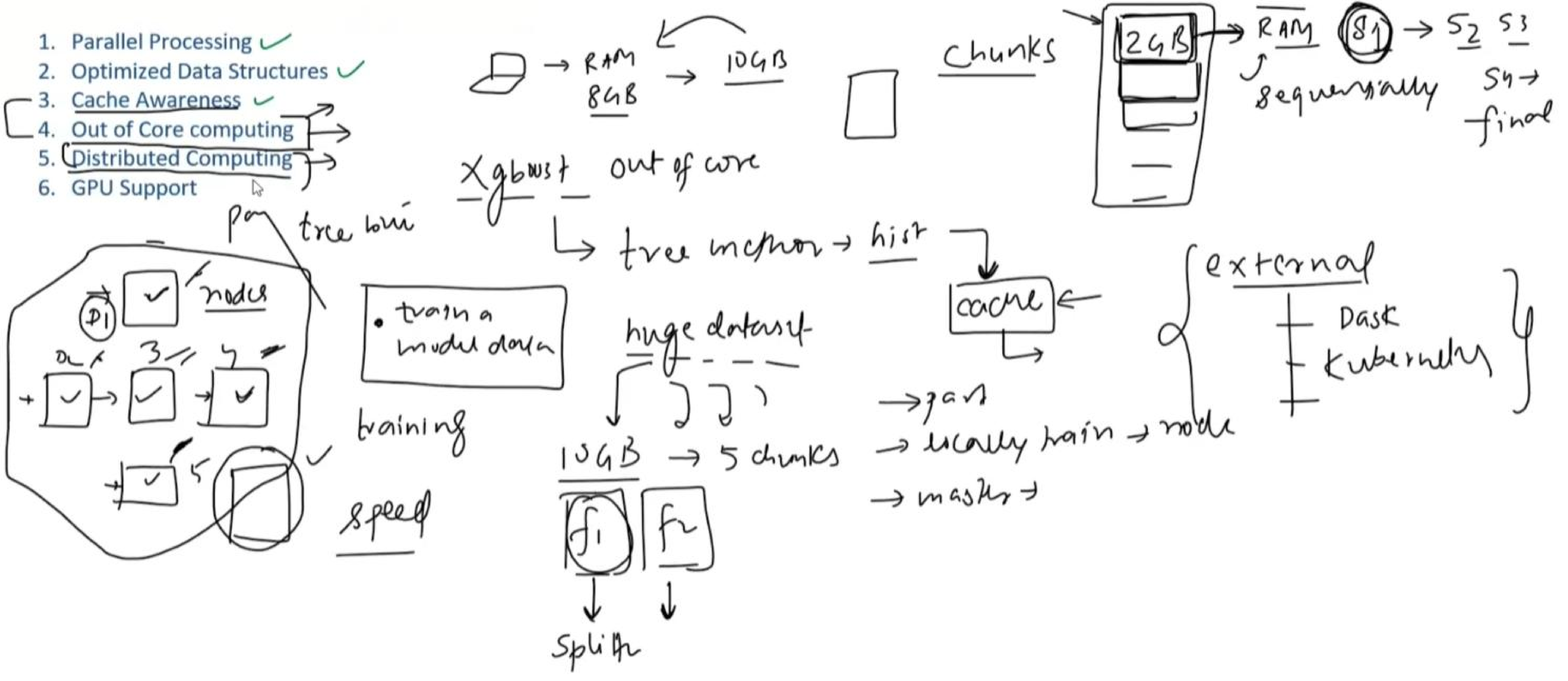
cache
fridge - RAM
cook → CPU

Xgboost

↓

Histogram based training

1. Parallel Processing
2. Optimized Data Structures
3. Cache Awareness
4. Out of Core computing
5. Distributed Computing
6. GPU Support



1. Parallel Processing ✓
2. Optimized Data Structures ✓
3. Cache Awareness ✓
4. Out of Core computing ✓
5. Distributed Computing ✓
6. GPU Support →

$\boxed{\text{CPU}}$ → data → CPU → $\boxed{\text{GPU}}$

↳ cores → less powerful

more in number

parallel

histogram

split finding

tree_method: gpu_hist

3. Performance →

14 October 2023 10:24

Xgboost

Xgboost → Loss function → min

1. Regularized Learning Objective
2. Handling Missing values
3. Sparsity Aware Split Finding
4. Efficient Split Finding(Weighted Quantile Sketch + Approximate Tree Learning)
5. Tree Pruning

$\boxed{L + reg}$

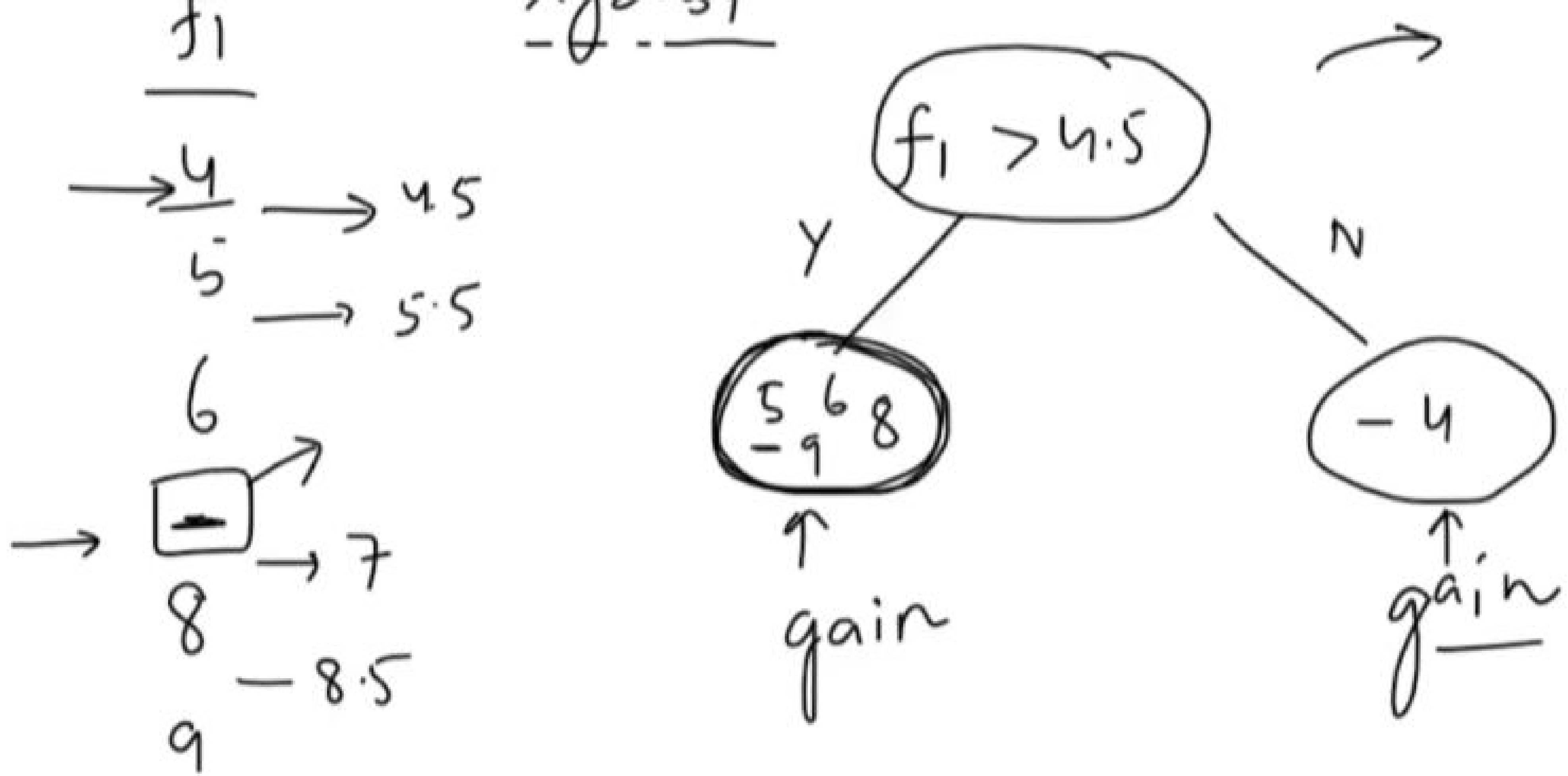
Linear reg
↓
Regular
(L1 & L2) → train ✓ → test X
overfitting

Gradient
(L) → reg
→ learning rate
→ tree construction

1. Regularized Learning Objective ✓
2. Handling Missing values
3. Sparsity Aware Split Finding
4. ~~Efficient Split Finding~~(Weighted Quantile Sketch + Approximate Tree Learning)
5. Tree Pruning

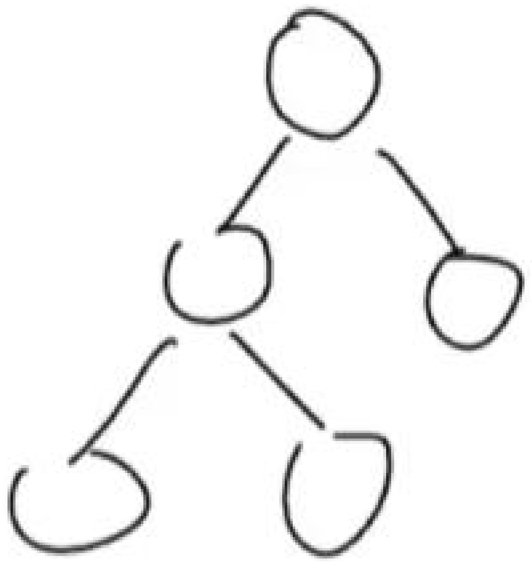
ml → missing
└→ prepro
└→ impute

xgboost



directional sense

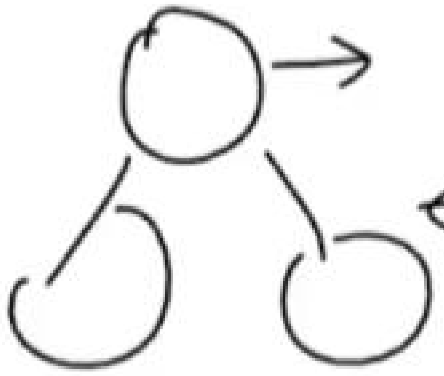
4. Efficient Split Finding(Weighted Quantile Sketch + Approximate Tree Learning)
5. Tree Pruning



1crore

f1

3 — 4
5 — 6
7 — 8
9
11
13



→ greedy search

slow

→ histogram based training

1 - 5 | 6 - 10 | 11 - 15 | . . .

discrete

0 - 30'm | 30'm - 45'm

bins

div

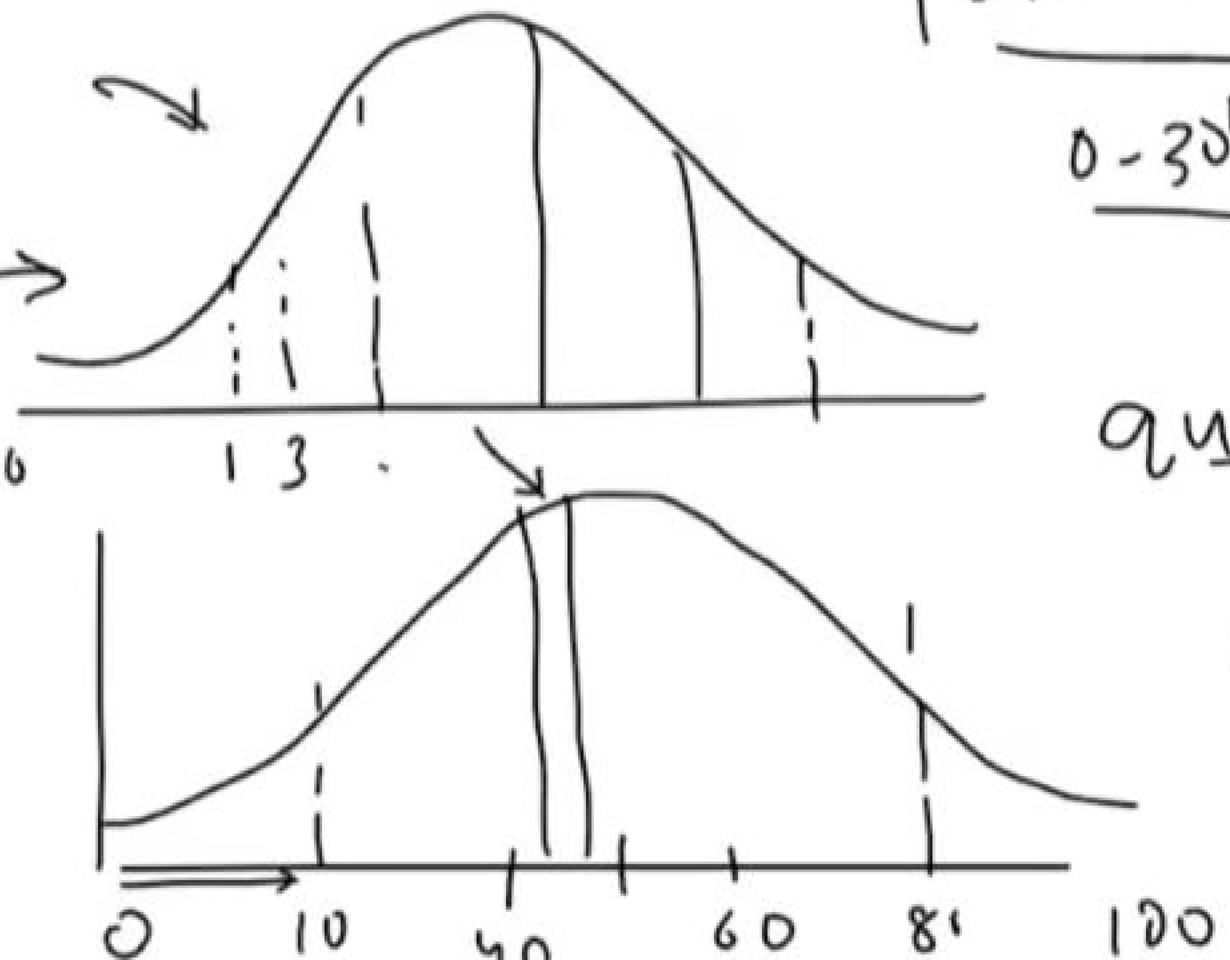
f1

$\boxed{0 - 100}$

↳ uniform

0 - 10 | 10 - 20 | 20 - 30 | . . 90 - 100

↳ unif



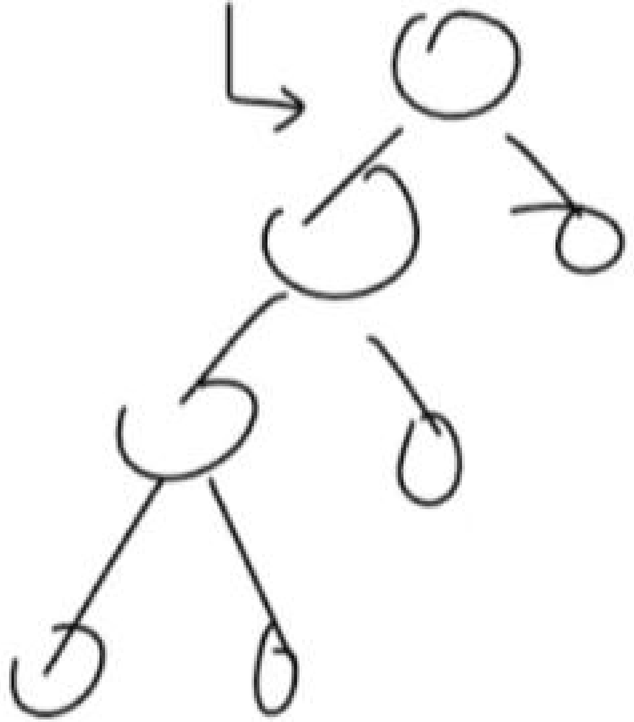
quantiles

↓

binning

GB

1. Regularized Learning Objective
2. Handling Missing values
3. Sparsity Aware Split Finding
4. Efficient Split Finding(Weighted Quantile Sketch + Approximate Tree Learning)
5. Tree Pruning → perform xgboost



→ reduce complexity → overfitting

Post pruning

prepruning

xgboost

prepruning

postpr

gamma value